DATA-260 Homework 7

DATA-260 Homework 7

GitHub repository: https://github.com/shrustishetty-2401/data260-9004

Student and Configuration

- Student: Shrusti Shetty
- SID4: 9004
- Port base: 8004
- Prefix: `s9004`
- Domain ID: 4
- Seed: 9004
- Verify seed: 269004
- Hardware: MacBook Pro, Apple Silicon
- Local model: `qwen3:8b`
- Branch: `hw7`
- Final tagged commit: `0af80386bf98421cdcab15176c2276a70a1e8dfe`

Part 1: JWT Authentication, Roles, and Linting

Implemented JWT authentication with MySQL-backed users.

Test users:

- `reader` / `data260`
- `writer` / `data260`
- `moderator` / `data260`

Verified:

- Valid login returns HTTP 200 and a JWT.
- Invalid login returns HTTP 401.
- Missing token returns HTTP 401.
- Invalid token returns HTTP 401.
- Reader package creation returns HTTP 403.
- Writer package creation returns HTTP 201.
- Writer package deletion returns HTTP 403.
- Moderator package deletion returns HTTP 200.
- Authenticated list/read request returns HTTP 200.

- Ruff and pre-commit checks pass.

Login Evidence



Login success and invalid login

Authentication Failure Evidence



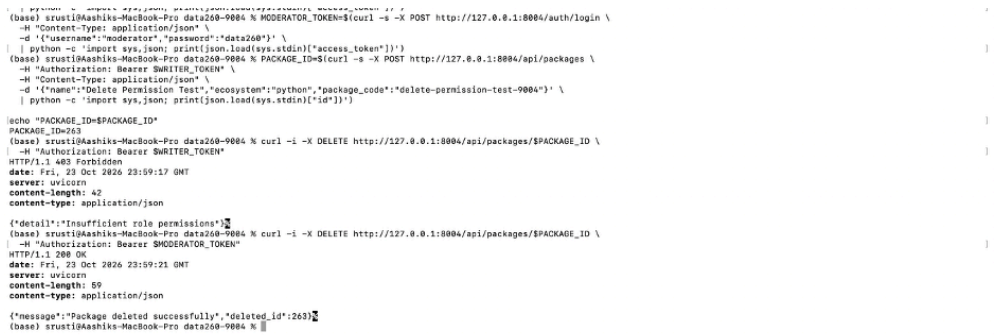
Missing and invalid token responses

Create Permission Evidence



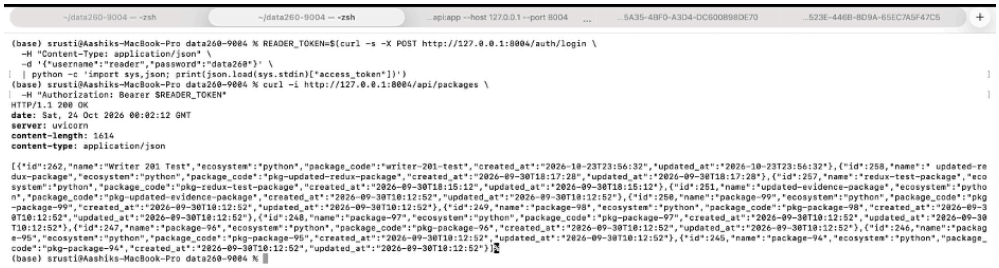
Reader 403 and writer 201 responses

Delete Permission Evidence



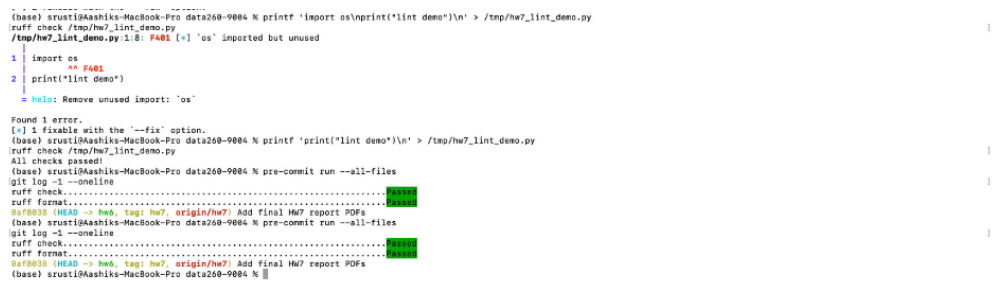
Writer 403 and moderator 200 responses

Authenticated Read Evidence



Authenticated list request

Linting Evidence



Ruff and pre-commit evidence

Part 2: Tool-Using Agent

Implemented a tool-using agent with:

- `search_vulnerabilities`
- `vulnerability_detail`
- `vulnerability_aggregate`

The agent supports:

- Multiple tool calls.
- Tool-role messages.
- Input validation.
- Malformed-argument repair.
- Unknown-tool blocking.
- Maximum-step limits.
- Safe handling of nonexistent vulnerability IDs.

Verification Tasks

Task A searched for a seeded vulnerability and retrieved the first result.

Task B retrieved an aggregate summary and searched for a supporting package example.

Task C searched for `package-0` and retrieved details for vulnerability IDs `10002` and `9752`.

```
(base) srusti@Aashiks-MacBook-Pro data260-9004 % PYTHONPATH=.:code python -c '
from agent_runner import run_agent

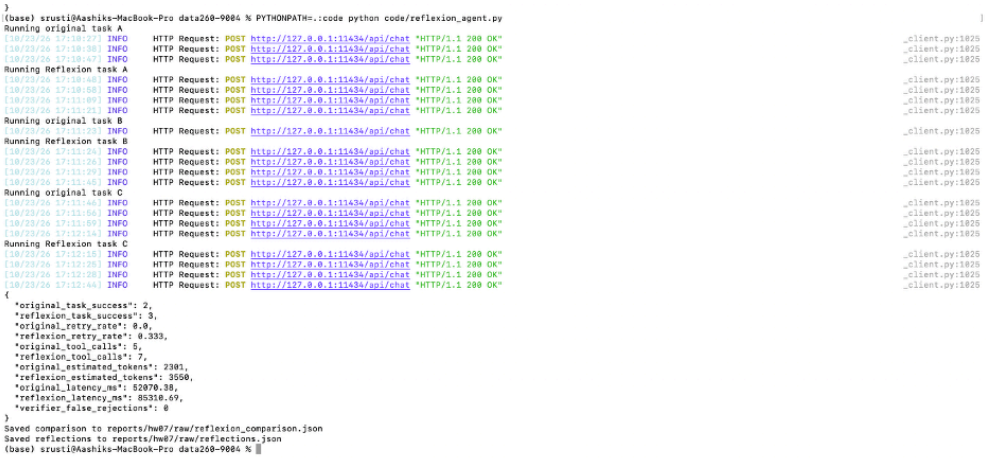
tasks = [
    (
        "A",
        "Search for a seeded vulnerability, then use the first result ID to retrieve its full details. You must make both tool calls before answering.",
        4,
    ),
    (
        "B",
        "First get an aggregate summary of the vulnerability database. Then search for package-0 to provide one concrete supporting example. You must make both tool calls before answering.",

        5,
    ),
    (
        "C",
        "Search for package-0 first. Then retrieve details for vulnerability IDs 10002 and 9752 and compare those two entities. You must make all three tool calls before answering.",
        6,
    ),
]

for name, prompt, ceiling in tasks:
    result = run_agent(
        prompt,
        max_steps=ceiling,
        log_path="reports/hw07/RUN_LOG.txt",
    )

    print(
        "TASK",
        name,
        "stop_reason=",
        result.get("stop_reason"),
        "tool_calls=",
        result.get("tool_calls"),
    )
'
[10/23/26 17:06:47] INFO     HTTP Request: POST http://127.0.0.1:11434/api/chat "HTTP/1.1 200 OK"                                                
```

Metric	Original loop	Reflexion loop
Task success	2	3
Retry rate	0.000	0.333
Tool calls	5	7
Estimated tokens	2278	3506
Latency	41793.48 ms	61113.38 ms
Verifier false rejections	0	0



Reflexion comparison

The verifier recorded zero false rejections. The stored reflections contain concise failure evidence and revised strategies without hidden chain-of-thought.

Evidence:

- `raw/reflexion_comparison.json`
- `raw/reflections.json`

Part 4: Citi Bike ReAct/MRKL Application

Implemented a React/Vite Citi Bike decision application with the following tools:

1. `csv_sql`
2. `policy_retriever`
3. `calculator`

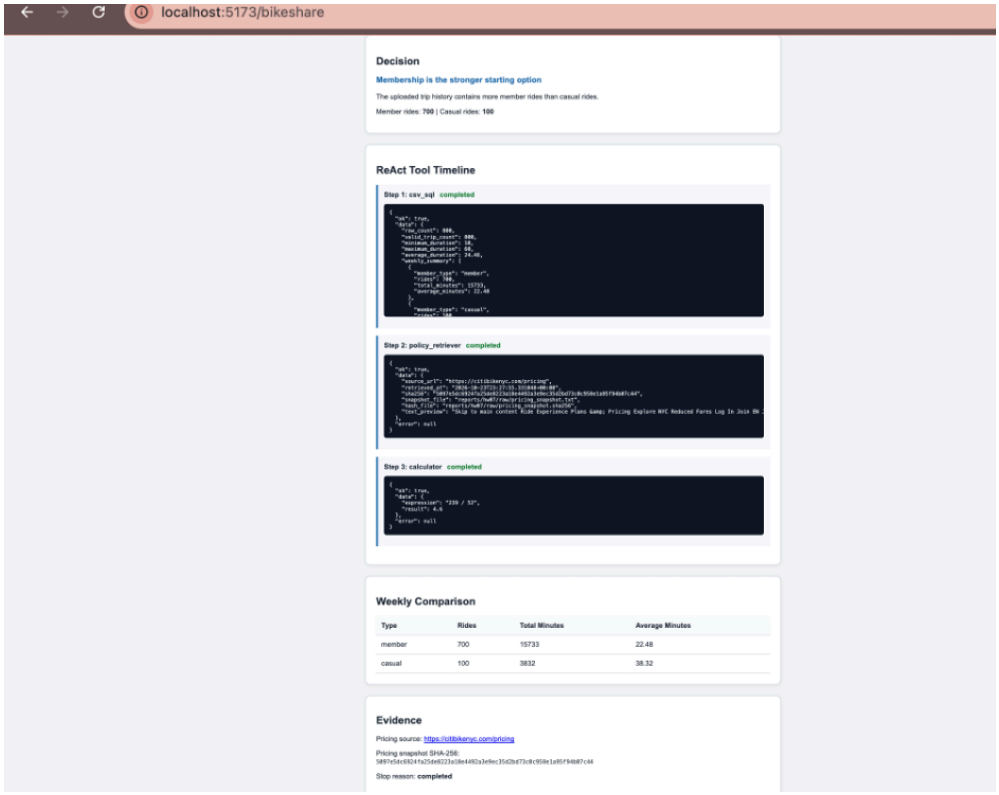
The application displays:

- Uploaded CSV filename.
- Tool execution timeline.
- Weekly member/casual comparison.
- Membership or pay-per-use decision.
- Pricing source.
- Pricing capture time.
- Pricing snapshot SHA-256 hash.
- Calculator output.
- Completion stop reason.

Dataset Results

Dataset	Rows	Member rides	Casual rides	Result
`bikeshare_membership_wins.csv`	800	700	100	Membership is stronger
`bikeshare_pay_per_use_wins.csv`	800	100	700	Pay-per-use is stronger

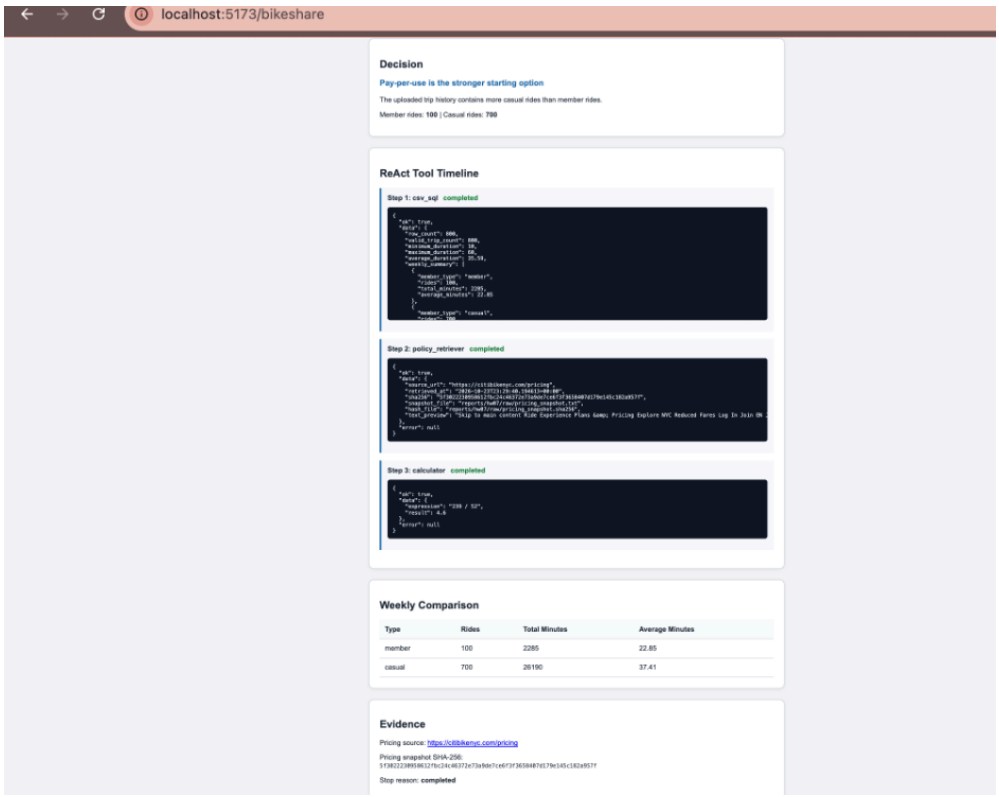
Membership-Win Scenario



Membership-win scenario

The membership-win dataset contains 700 member rides and 100 casual rides. The application selected membership because member usage was dominant.

Pay-Per-Use-Win Scenario



Pay-per-use-win scenario

The pay-per-use-win dataset contains 100 member rides and 700 casual rides. The application selected pay-per-use because casual usage was dominant.

Pricing evidence:

- `raw/pricing_snapshot.txt`
- `raw/pricing_snapshot.sha256`

Part 5: Prompt-Injection Defense

Threat Model

The threat model covers:

- Recommendation override: an attacker attempts to force an incorrect plan recommendation.
- Non-read-only tool misuse: an attacker attempts to make the SQL tool modify data.
- Data exfiltration: an attacker attempts to include restricted uploaded content in the final answer.

The hostile fixture is stored at:

- `raw/hostile_pricing_fixture.html`

Tested attacks:

- Instruction override.
- Unauthorized tool abuse.
- CSV-based prompt injection.

Results:

- Attack cases: 3
- Defended cases: 3
- Defense success rate: 100%



Prompt-injection defense

Retrieved text is treated as untrusted data. Tool-boundary enforcement is stronger than prompt-only defense because the tool itself rejects unsafe operations even if an input attempts to bypass the instructions.

Evidence:

- `raw/prompt_injection_results.json`
- `raw/hostile_pricing_fixture.html`

Part 6: File Tools and Safety Gate

Implemented:

- `read_file`
- `write_file`
- `edit_file`
- `glob_files`
- `search_in_files`
- `bash`

The safety gate includes:

- `SAFE_PREFIXES`
- `is_destructive`
- `confirm_bash`
- Explicit confirmation for destructive commands.
- Path traversal protection.

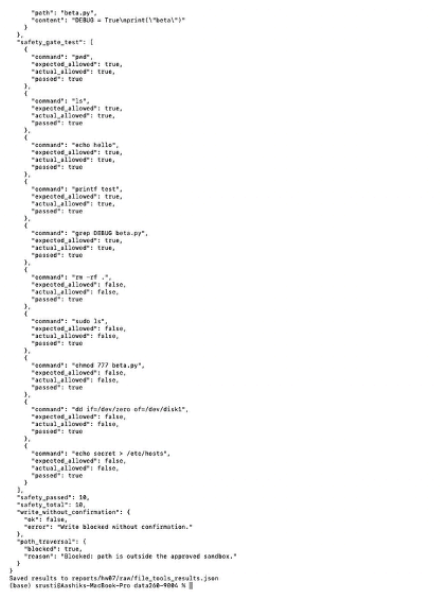
- Unconfirmed write protection.

The integration test performed:

`glob -> search -> read`

The safety suite tested ten commands. Five safe commands were allowed and five destructive or restricted commands were blocked.

The results confirmed that file listing, file search, file reading, unconfirmed-write blocking, path-traversal blocking, and destructive-command confirmation all work as intended.



File tools and safety evidence

Evidence:

- `raw/file_tools_results.json`

Reproducibility Evidence

- `RUN_LOG.txt`
- `METRICS.md`
- `AI_USE.md`
- `verification.json`
- `raw/agent_reliability.json`
- `raw/reflexion_comparison.json`
- `raw/reflections.json`
- `raw/prompt_injection_results.json`
- `raw/file_tools_results.json`
- `raw/pricing_snapshot.txt`
- `raw/pricing_snapshot.sha256`
- `raw/hostile_pricing_fixture.html`
- `raw/bikeshare_membership_wins.csv`
- `raw/bikeshare_pay_per_use_wins.csv`

Final Verification

The following checks passed:

- Python compilation.
- Ruff check.
- Pre-commit hooks.
- Backend API startup.
- JWT authentication tests.
- Invalid-login test.
- Missing-token test.
- Role-permission tests.
- Authenticated read test.
- Agent tests.
- Tool tests.
- Reliability experiment.
- Reflexion experiment.
- Bike-share API test.
- React production build.
- Prompt-injection tests.
- File-safety tests.

The final `hw7` branch and tag point to:

```
0af80386bf98421cdcab15176c2276a70a1e8dfe
```

Submission Files

The final submission PDF is:

```
reports/hw07/Shetty_HW7.pdf
```

The report includes screenshots for Parts 1 through 6, machine-readable experiment outputs, verification results, and reproducibility evidence.